

AI Product Comparison Engine

Technical Handover Document

Cross-encoder product matching with embedding retrieval. This document records the current state, every measurement taken, the reasoning behind each major decision, and what remains. It is written to be read cold, by someone with no prior context.

Status	Working end-to-end; benchmark-competitive; not production-hardened
Model	microsoft/deberta-v3-small cross-encoder, binary (same / different product)
Headline result	Beats published Ditto & RoBERTa baselines on 5 of 6 benchmarks
Retrieval	95.7% recall@50 over 112,746 products at 24 ms
Tests	45 passing, 1 known failure (documented below)
Best checkpoint	trained_model_real/ (binary, 'colval' serialization)
Date	30 July 2026

THE ONE THING TO KNOW. Every accuracy problem in this project turned out to be a data problem, never a model problem. The same architecture scored 45.9 F1 trained on rule-generated labels and 75.6 trained on human labels. Before changing the model, check the data.

1. Results

1.1 Benchmark performance (supervised, comparable to published work)

One model trained once on a merged corpus, evaluated per benchmark. The published baselines used a *separately trained model per benchmark*, which makes matching them harder, not easier. 'argmax' is the honest number; 'best-thr' is tuned on the test set and is an optimistic upper bound.

Benchmark	argmax	best-thr	Published	Delta
Structured_Amazon-Google	78.57	79.12	Ditto 75.58	+2.99
Structured_Walmart-Amazon	84.38	85.64	Ditto 86.76	-2.38
Textual_Abt-Buy	89.36	90.28	Ditto 89.33	+0.03
WDC-SEEN	80.77	81.82	RoBERTa 78.58	+2.19
WDC-HALF-SEEN	80.51	81.50	RoBERTa 75.91	+4.60
WDC-UNSEEN	74.78	75.86	RoBERTa 71.14	+3.64

Caveat on three numbers. The Ditto ER-Magellan figures (75.58 / 86.76 / 89.33) are hardcoded in `train_on_real_corpus.py` from a secondary source and were never verified against the paper (the PDF fetch failed). The WDC / RoBERTa figures WERE verified from arxiv 2301.09521 Table 3 - and those are the ones beaten by the widest margins.

1.2 Generalisation to unseen products

The drop from SEEN to UNSEEN measures whether a matcher handles products it has never encountered - the property that matters most for a live catalog.

System	SEEN -> UNSEEN drop
This model	-5.99 (best)
RoBERTa (published)	-7.44
Ditto (published)	-8.92
R-SupCon (contrastive, published)	-24.65

R-SupCon has the best SEEN score of any published system and the worst UNSEEN score. This is direct evidence **against** adopting contrastive learning here.

1.3 Retrieval at scale

Catalog	112,746 products
Index	FAISS IndexFlatIP (exact, no approximation), 165 MB
recall@1	25.7%
recall@10	75.4%
recall@50	95.7% <- top-50 is 0.04% of the catalog
recall@100	97.6%
Latency	24 ms p50 / 28 ms p95 (CPU)

recall@1 of 25.7% alongside recall@50 of 95.7% is the whole argument for two stages: the bi-encoder is a poor ranker but an excellent filter. Recall@k is the ceiling on the entire system - a match never retrieved is never re-ranked.

1.4 Real-world validation (Indian e-commerce)

14 pairs from live Amazon India / Google Shopping data, hand-labelled with written justifications. Small sample - directional, not a measurement.

Metric	Before serialization fix	After
Accuracy	79%	86%
Recall on true matches	0%	100%
Precision	n/a (no positives)	60% (3 TP of 5)

Also measured: on 137 cross-merchant pairs Google Shopping claims are the same product, the model agreed 85% of the time, and its disagreements were mostly **Google's errors** - independently reproducing hand labels on SanDisk Ultra vs CZ48, ColorFit Pulse 2 vs Icon 2, and Milton 900ml vs 520ml. On 150 same-brand different-model-number pairs the false-positive rate was 1%.

2. Key findings - read this before changing anything

These took days to establish. Each is a measurement, not an opinion, and each closes off a line of work that looks attractive but is already known to be wrong.

2.1 The labels were the bottleneck, not the model

The original dataset was generated by rules in `generate_relationship_pairs.py`. The model learned to reproduce the rule and scored 95% - a number that measured nothing. Graded against human-labelled WDC pairs the same checkpoint scored **45.9 F1**. Retraining the identical architecture on 38k human labels gave **75.6**. Nothing about the model changed.

2.2 Entity leakage barely mattered - and why that is informative

29.85% of test rows contained a product seen in training. Fixing it (entity-level split via union-find over the product graph) moved accuracy by **0.07 percentage points** (0.9508 -> 0.9501). That should be impossible unless the model was never memorising entities. It wasn't: it was recovering the labelling rule, which generalises perfectly to unseen products. When labels come from a rule, leakage is nearly irrelevant.

2.3 A serialization mismatch silently destroyed all recall

The model is trained on `COL brand VAL ... COL title VAL ...`, but `inference.py` fed it `title | brand x | description`. Nothing raised. Three unmistakable matches scored 49.8% / 20.6% / 4.4% instead of ~100%, recall on real matches was **zero**, and `/health` reported 'ok' throughout.

Pair	pipeline format	COL/VAL format
boAt Airdopes 141 Elite ANC (x2 phrasings)	20.6%	99.9%
JBL Tune 215BT (x2 phrasings)	4.4%	100.0%

Now guarded: the serialization is recorded inside the checkpoint (`training_metadata.json` -> "serialization") and `ProductComparer` reads it, warning loudly when absent. Note the bi-encoder deliberately uses plain text instead - two models, two formats, both correct. Do not unify them.

2.4 WEAKLY_SIMILAR was never a real class

An audit of all 5,916 rows carrying that label found only **712 (12%)** had any measurable justification, and just **2** came from a form-factor mismatch - both junk. 81% came from a per-category *relative percentile* rule; other rows came from an absolute Jaccard threshold and from a structural rule. Three incompatible definitions, one label. Attempts to build clean replacement data topped out at 438 rows with ~17% contamination. The class was dropped; the task is now binary.

2.5 Serialization format made no measurable difference to accuracy

A/B test of Ditto-style COL/VAL versus the project's own format: **+0.10 F1 mean**, with the sign flipping across splits. Within noise. (This is separate from 2.3 - there the problem was train/inference *mismatch*, not the choice of format.)

2.6 Google Shopping clustering is not ground truth

It grouped 'Thirty First Earbuds **For** Boat 141' - a third-party item at roughly double the price - with the genuine boAt product, and placed one identical Amazon listing into two different colour clusters simultaneously. Measured disagreement with hand labels: **40%** of its claimed matches. Useful as high-recall candidates; unusable as labels.

3. Architecture

```
POST /search {"product": {"title": "..."}}
|
+--> Stage 1 bi-encoder (all-MiniLM-L6-v2) + FAISS over full catalog
| plain text | 24 ms | 95.7% recall@50
+--> Stage 2 cross-encoder (deberta-v3-small) re-ranks ~50 candidates
| COL/VAL text | batched, one forward pass per batch
+--> exact_match + ranked alternatives
```

Neither stage works alone. The cross-encoder is too slow to score a catalog (~5-10 ms/pair on GPU means over an hour for 500k). The bi-encoder is too weak to pick the winner (25.7% recall@1). This retrieve-then-rerank split is the standard production architecture and is what the cross-encoder was always for.

3.1 API surface

Endpoint	Purpose	Notes
POST /search	Find a product in the indexed catalog	Both stages. Needs an index; 503 if absent
POST /compare	Score a caller-supplied shortlist	No index needed. Max 200 candidates (413)
GET /health	Model + index status	Reports model_loaded, num_labels, catalog_size

3.2 Key files

File	Role
exact_match/inference.py	ProductComparer: compare(), score_pairs() (batched), serialization handling
exact_match/train.py	Training loop (unmodified; drivers configure it from outside)
exact_match/preprocessing.py	Cleaning + ENTITY-LEVEL split (union-find over product graph)
ranking/embedding_retrieval.py	ProductIndex: build/save/load/search, recall@k evaluation
ranking/ranker.py	Two-stage orchestration; handles binary AND 5-class models
build_real_corpus.py	Merges WDC + ER-Magellan into one training corpus, with leakage checks
train_on_real_corpus.py	Trains on the merged corpus; evaluates per benchmark
finetune_on_wdc.py	WDC-only fine-tuning with published hyperparameters
evaluate_on_wdc.py	Zero-shot evaluation against WDC gold standards
collect_amazon_listings.py	Real Amazon India listings via RapidAPI (key from env)
collect_shopping_offers.py	Cross-merchant offers via Google Shopping API
build_label_candidates.py	Builds unlabelled candidate pairs for human adjudication
audit_weakly_similar.py	Traces why each row carries its label

3.3 Training configuration that produced the best model

```
base microsoft/deberta-v3-small (141.9M params)
corpus 38,145 human-labelled pairs (WDC 19,607 + Amazon-Google 6,674
+ Walmart-Amazon 6,142 + Abt-Buy 5,722)
labels binary (2)
serialization COL brand VAL ... COL title VAL ... COL description VAL ...
lr 5e-5 effective batch 64 (32 x 2 accum) warmup 0.05 fp16
epochs 15, early-stopped at 5 (best epoch 4) ~40 min on a T4
```

4. Test results

46 collected, 45 passing, 1 known failure. All use stubs - no trained model or network required - so the suite runs in about 15 seconds.

Test file	N	Covers
tests/test_split_leakage.py	7	Entity-level split; verified to FAIL against the old row-level split
tests/test_ranker_binary.py	8	Binary model path, batching, 413/503 handling
tests/test_search_endpoint.py	8	/search wiring, retrieve_k pass-through, failure modes
ranking/test_ranking.py	6	5-class ranking logic, category filtering, sort order
tests/test_api.py	3	/compare shape and validation
tests/test_data_quality.py	5	Contradiction rules, severity filtering
tests/test_generate_relationship_pairs.py	5	Labelling heuristic (1 FAILING - see below)
tests/test_utils.py	5	format_time

4.1 The known failure

`test_same_category_low_overlap_is_weakly_similar` expects `WEAKLY_SIMILAR` for 'boAt Airdopes 300 / basic model' vs 'Sony WF earbuds / premium noise cancelling flagship'. It now returns `SIMILAR_ALTERNATIVE`.

This is **not** a regression - the test asserts the old Jaccard behaviour that was deliberately removed. But it exposes a real gap: tier detection reads only *numeric* specs, so a tier stated in words ('basic' vs 'premium flagship') is invisible. Either add lexical tier detection or mark the test xfail with a reason. A permanently red test trains people to ignore red tests.

4.2 Dangerous files in tests/

tests/test_1.py has 0 assertions and is a data-merging script that **WRITES** to `data/`. If CI ever runs bare `pytest`, collection executes it and corrupts your data. `tests/test_example.py` and `test.py` are also 0-assertion manual scripts. All three should be moved out of `tests/` or deleted.

5. What remains

5.1 Blocking real deployment

Item	Detail
No authentication	Verified: zero matches for <code>api_key</code> / <code>rate_limit</code> / <code>authenticate</code> in <code>api/</code> . Anyone reaching the port gets unlimited GPU inf.
No rate limiting	Same. <code>/compare</code> caps candidates at 200 and <code>/search</code> caps <code>retrieve_k</code> at 200, but there is no per-client limit.
Index cannot refresh	Rebuild is all-or-nothing, ~16 min for 112k on CPU. A live catalog needs incremental add/remove.
Fake test files	<code>tests/test_1.py</code> writes to <code>data/</code> during collection (see 4.2).
No Dockerfile	No deployment or container config exists.

5.2 Accuracy levers (optional, diminishing returns)

Lever	Expected	Effort	Notes
~2,000 Indian labelled pairs	Largest real gain	~2 sessions	193 candidates already built; 14 labelled. Targets confusable SKUs
Per-category threshold calibration	+1-2 F1	~1 hour	Optimal threshold varied 0.20-0.80 across benchmarks
deberta-v3-base	+1-3 F1	~1 day	3x params; current model already matches roberta-base

LSPC 219k corpus	+1-3 F1	~4.5 h	add_lspc_corpus.py written; leakage check NOT YET RUN - run --check-only
------------------	---------	--------	--

5.3 Technical debt

Item	Detail
README is stale	163 lines with ZERO mentions of search, FAISS, retrieval, trained_model_real or er_magellan. Still documents a 2-c
config.py defaults wrong	RAW_DATA_PATH still points at products_structured.csv; NUM_LABELS=2 left over from the abandoned pipeline.
Dead synthetic generators	generate_dataset.py (683 lines, contains unreachable code), generate_dataset_chunked.py, generate_synthetic_pa
save_model.py label_map bug	Writes a hardcoded binary label_map regardless of NUM_LABELS. Harmless while the model is binary; wrong if a 5

5.4 Unvalidated

GPU latency (only CPU measured) | concurrent request handling (never load-tested) | catalog scale beyond 112,746 products.

6. Traps that have already cost time

Each of these caused a real failure during development. They are listed because none of them announce themselves.

6.1 config must be set BEFORE importing exact_match

model.py and save_model.py read config values as **default argument values**, which Python binds once at import. Setting config.NUM_LABELS afterwards is silently ignored and you get a 2-class model when you wanted 5. Verified empirically: wrong order gives 2, correct order gives 5. All driver scripts override config first, then import.

6.2 .gitignore contains *.txt

This silently swallowed all 41,519 ER-Magellan benchmark pairs. `git add` reported nothing, the push looked clean, and training failed on the remote with 'No datasets found'. A negation rule (`!data/er_magellan/**/*.txt`) now exists. An old `/api` rule likewise left `api/schemas.py` untracked, so a fresh clone could not import the API at all.

6.3 Kaggle decompresses uploaded archives

Files uploaded as `.json.gz` arrive as `.json`. Loaders must accept both. Kaggle also wipes `/kaggle/working` between sessions, and `/kaggle/input` is read-only - never extract into it.

6.4 tqdm floods Kaggle logs

Under `!python` the progress bar cannot rewrite its line, so every step prints a new row and buries the epoch summaries. Driver scripts disable it by default (`--progress` re-enables).

6.5 Zip the right model directory

`trained_model/` was the OLD 5-class model (45.9 F1). It has now been deleted and the API fallback to it removed - serving a broken model while reporting healthy is worse than failing loudly. The good checkpoint is **`trained_model_real/`**.

7. Reproducing the current model

```
# 1. build the corpus (needs data/er_magellan + WDC 50pair files)
python build_real_corpus.py --wdc /path/to/wdc --wdc-size large

# 2. train (~40 min on a T4)
python train_on_real_corpus.py --wdc /path/to/wdc

# 3. build the retrieval index
python -m ranking.embedding_retrieval build --catalog <catalog.jsonl> --out data/product_index

# 4. measure recall BEFORE trusting it
python -m ranking.embedding_retrieval evaluate-pairs --index data/product_index \
    --ground-truth data/scale_ground_truth.csv

# 5. serve
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Data sources (all free, human-labelled):

Source	Pairs	Where
WDC Products 2023	~20k train + 3 x 4,500 gold	datacommons.org/largescaleproductcorpus/wdc-products/
ER-Magellan (Abt-Buy, Amazon-Google, Walmart-Amazon)	61,277	github.com/megagonlabs/ditto -> <code>data/er_magellan/</code>
WDC LSPC 2017 (unused)	219,135 at xlarge	huggingface.co/datasets/wdc/products-2017

End of handover document. Generated 30 July 2026.